

In HTML, the `<form>` tag is used to nest input forms. The `method` attribute defines which HTTP method to be used for input data transfer. For input forms, the **POST method** is usually used.

The `<input>` tag with the `type="submit"` attribute creates a data submission button. You can also use the `<button>` tag instead of the `<input>` tag.

2. {{ form.as_p }}

The `form` part is an object defined by the view. In the `employee_learning` app case, we set the `field` attributes like the one below.

views.py

```
fields=('title', 'level', 'employee')
```

Those model fields are converted into input forms through the `form` object.

The `as_p()` method is used to add the `<p>` tags in each form element. There are other similar methods like `as_ul()`, `as_table()`, but `as_p()` that are often used for the form object.

3. {% csrf_token %}

This is used to protect against **Cross-Site Request Forgery (CSRF)**. CSRF is a type of cyberattack that forces an end user to execute unwanted actions on a web application, for example, by sending JavaScript code through the input forms. By adding `{% csrf_token %}`, Django generates a token to build a more secure communication when a user is sending data through the input forms.

If you don't add this tag, when a user submits their input, the user will see the *Forbidden* error message like the one below.